

Drug Administration Analysis in Excel

This guide replicates the Python notebook logic in Excel. The goal is to classify each dose row by drug subtype and calculate total ordered amounts, using a three-step fallback to resolve the dose for each row.

Requires Excel 365 or Excel 2021. The formulas use `XLOOKUP` , `XMATCH` , `IFS` , `TEXTBEFORE` , and `TEXTAFTER` . Nothing here needs Ctrl+Shift+Enter.

Working in Google Sheets instead? Two formulas need one change. Excel evaluates a function fed a multi-cell range as an array automatically; **Sheets does not** — it silently uses only the first cell of the range. Where that matters, the step shows a **Google Sheets** variant wrapped in `ARRAYFORMULA(...)` . Use that form in Sheets, and the plain form in Excel. Steps 5 through 7 are identical on both.

The symptom when this bites, in case you hit it elsewhere: the inner test looks correct and returns a single `TRUE` , but `XLOOKUP` reports *"Array arguments to XLOOKUP are of different size"* — one element on one side, the full range on the other.

The design principle throughout: **drug names, units, and conversion factors live in tables, not in formulas**. Adding a drug or a unit means adding a row to the `Reference` sheet. You should never have to edit a formula to onboard new data.

Column Reference

After importing the CSV (see Step 1), the columns are:

Column	Name
A	Start Date
B	End Date
C	Slices by Component Simple Generic Name
D	Order Name

Column	Name
E	Ordered Dose Amount
F	Ordered Dose Unit
G	Administration Instant
H	Administered Dose Amount (do not use)
I	Administered Dose Unit (do not use)

Important: Columns H and I are checker columns for verification only. All dose logic uses columns D, E, and F.

Helper columns you will add run from **J to P**:

Column	Header	Purpose
J	Drug Type	Which drug subtype this row is
K	Name Unit	Unit found at the end of the Order Name
L	Name Amount	Number that precedes that unit
M	Dose Source	Which fallback step resolved this row
N	Resolved Unit	The unit actually used
O	Resolved Amount	The amount actually used
P	Converted Amount	Resolved Amount converted to the drug's target unit

Step 1: Open and Prepare the Data

1. Open `rose_data.csv` in Excel.
2. The first 10 rows are report metadata. Delete rows 1–10 so that the column headers (`Start Date` , `End Date` , etc.) are in **row 1** and data begins in **row 2**.
3. Freeze row 1 (**View** → **Freeze Top Row**) and turn on filtering (**Data** → **Filter**).

Do not use Home → **Format as Table**. Converting the range to an Excel Table rewrites cell references into structured form (`[@[Order Name]]` instead of `D2`), and

every formula in this guide is written in plain A1 style. Freeze and filter give you the same convenience without the rewriting.

Step 2: Set Up the Reference Sheet

Create a new sheet named `Reference`. It holds four small tables. Everything the formulas need to know about drugs, units, and defaults lives here.

Leave one blank column between tables so nothing collides.

Table 1 — Drug Keywords (A1:B12)

Each row maps a keyword to a drug type. **Order matters** — the first keyword found in the Order Name wins, so keep the sequence below.

keyword	drugtype
albumin human 5 %	Albumin 5%
albumin human 25 %	Albumin 25%
balfaxar	Balfaxar (4F-PCC)
prothrombin complex human-lans	Balfaxar (4F-PCC)
kcentra	Kcentra (4F-PCC)
antithrombin	Antithrombin III (Thrombate)
alphanate	Antihemophilic Factor-VWF (Alphanate)
rahf-pfm	Antihemophilic Factor rAHF-PFM (Advate)
advate	Antihemophilic Factor rAHF-PFM (Advate)
coagulation factor viia	Coagulation Factor VIIa (NovoSeven)
coagulation factor ix	Coagulation Factor IX (Benefix)

Two drugs have two keywords each — Balfaxar and Advate — which is why this is a table of 11 rows and not 9. To add a new alias, add a row.

Define these names (**Formulas** → **Define Name**):

- `kw_text` → `Reference!$A$2:$A$12`
- `kw_type` → `Reference!$B$2:$B$12`

You can find screenshots in the screenshots folder of where to rename these areas. If you do not like this, you can simply replace any mention of these names with the formulation itself (i.e., replace `kw_text` with `Reference!$A$2:$A$12` in the formula)

Table 2 — Unit Conversions (D1:G15)

Column G is a **key column** that joins the drug type and unit into one lookup value. This is what lets every later formula use a plain `XLOOKUP` instead of an array formula.

In **G2** enter `=D2&"|"&E2` and fill down. Do not type the keys by hand.

The formula has only two moving parts, but it is worth being precise about them because four later formulas depend on it:

Piece	On its own	What it contributes here
D2 , E2	The drug type and source unit on this row.	The two things that together identify a conversion factor. Neither is unique alone — <code>Units</code> appears against five drugs, and <code>Alphanate</code> appears against three units.
&	Joins two pieces of text into one.	Collapses that two-part identity into a single value, so it can be matched with one lookup instead of two coordinated ones.
the pipe separator	The single literal character sitting between the two joined values.	Chosen because it never occurs in a drug name or a unit. Without a separator, <code>Albumin 5% + g</code> and <code>Albumin 5 + %g</code> would collapse into the same key.

Why bother: matching on two columns at once normally forces an array formula — the kind needing `Ctrl+Shift+Enter`, which is exactly what made the previous version of this guide hard to read and fragile across Excel versions. Doing the join once, here, in a real column of cells, means every lookup later is an ordinary one.

drugtype	sourceunit	factor	key (formula)
Albumin 5%	g	1	(formula above)
Albumin 5%	mL	0.05	(fill down)
Albumin 25%	g	1	(fill down)
Albumin 25%	mL	0.25	(fill down)
Balfaxar (4F-PCC)	Units	1	(fill down)
Kcentra (4F-PCC)	Units	1	(fill down)
Antithrombin III (Thrombate)	Units	1	(fill down)
Antihemophilic Factor-VWF (Alphanate)	VWF:RCo Units	1	(fill down)
Antihemophilic Factor-VWF (Alphanate)	Int'l Units	1	(fill down)
Antihemophilic Factor-VWF (Alphanate)	Units	1	(fill down)
Antihemophilic Factor rAHF-PFM (Advate)	Units	1	(fill down)
Coagulation Factor VIIa (NovoSeven)	mg	1	(fill down)
Coagulation Factor VIIa (NovoSeven)	mcg	0.001	(fill down)
Coagulation Factor IX (Benefix)	Units	1	(fill down)

Define:

- conv_key → Reference!\$G\$2:\$G\$15
- conv_factor → Reference!\$F\$2:\$F\$15

You will also find screenshots of these in the screenshots folder if you still need help with this technique.

On weight-based units: g/kg , Units/kg , mcg/kg , and similar are deliberately **absent** from this table. They cannot be converted without patient weight. Leaving them out is what makes those rows fall through to the name-extraction step, which is the intended behavior.

Table 3 — Default Doses (I1:K10)

drugtype	defaultamount	defaultunit
Albumin 5%	(fill in)	g
Albumin 25%	(fill in)	g
Balfaxar (4F-PCC)	(fill in)	Units
Kcentra (4F-PCC)	(fill in)	Units
Antithrombin III (Thrombate)	(fill in)	Units
Antihemophilic Factor-VWF (Alphanate)	(fill in)	Units
Antihemophilic Factor rAHF-PFM (Advate)	(fill in)	Units
Coagulation Factor VIIa (NovoSeven)	(fill in)	mg
Coagulation Factor IX (Benefix)	(fill in)	Units

Define:

- `def_type` → `Reference!$I$2:$I$10`
- `def_amt` → `Reference!$J$2:$J$10`
- `def_unit` → `Reference!$K$2:$K$10`

Until you fill these in, any row whose Dose Source is `Default` contributes **0** to the totals, because a blank cell reads as zero. Step 7 shows you how to count those rows so you know how much is riding on them.

Table 4 — Units Found in Order Names (M1:M8)

The units that can appear at the end of an Order Name, **longest first**. The order is what makes multi-word units work: `VWF:RCo Units` must be tested before `Units`, or the match would stop at the last word and lose the prefix. Likewise `mcg` before `mg` before `g`.

nameunit
VWF:RCo Units
Int'l Units
Units

nameunit
mcg
mg
mL
g

Define:

- `name_units` → `Reference!$M$2:$M$8`

Step 3: Drug Type Classification (Column J)

Add a header `Drug Type` in **J1**. In **J2**:

```
=IFNA(XLOOKUP(TRUE, ISNUMBER(SEARCH(kw_text, D2)), kw_type), "Unmatched")
```

In **Google Sheets**, wrap the array-producing part in `ARRAYFORMULA` so all 11 keywords are tested instead of just the first:

```
=IFNA(XLOOKUP(TRUE, ARRAYFORMULA(ISNUMBER(SEARCH(kw_text, D2))), kw_type), "Unmatched")
```

Fill down for all rows.

Piece by piece:

Piece	On its own	What it contributes here
<code>SEARCH(kw_text, D2)</code>	Finds where one string starts inside another, case-insensitively. Returns a character position, or <code>#VALUE!</code> if the string isn't there.	<code>kw_text</code> is 11 cells, so this performs 11 searches at once and returns 11 answers — a position for every keyword present in the Order Name, an error for every one that isn't.
<code>ISNUMBER(...)</code>	Returns <code>TRUE</code> for a number and <code>FALSE</code> for anything else,	Flattens that mixed array into 11 clean flags meaning <i>did this keyword</i>

Piece	On its own	What it contributes here
	including errors.	<i>hit?</i> It is also what absorbs the <code>#VALUE!</code> results, so they never reach the outer functions.
<code>XLOOKUP(TRUE, <flags>, kw_type)</code>	Searches a range for a value and returns the entry in the same position of a second range.	Finds the first <code>TRUE</code> and returns the drug type beside it. First-match is not incidental — it is what makes Table 1's row order the precedence order.
<code>IFNA(..., "Unmatched")</code>	Supplies a fallback when the expression inside it returns <code>#N/A</code> — and only <code>#N/A</code> . Any other error passes straight through.	<code>XLOOKUP</code> returns <code>#N/A</code> when no flag was <code>TRUE</code> , and this turns that into the readable label "Unmatched". Deliberately not <code>IFERROR</code> : see the warning below.

Together: the two inner functions convert *"does this name contain any of my keywords?"* into a list of yes/no answers; the two outer functions take the first yes and name it, or say plainly that there wasn't one.

Why `IFNA` **and not** `IFERROR`. `IFERROR` swallows *every* error class — `#VALUE!`, `#NAME?`, `#REF!` — and would report all of them as "Unmatched". A broken named range would then be indistinguishable from a name that genuinely matched no keyword, and you would go looking for a missing keyword that isn't the problem. `IFNA` traps only `#N/A`, which is the one error that actually means "no match found". Everything else stays visible.

If J shows `Unmatched` **while the inner** `ISNUMBER(...)` **shows** `TRUE`, the two halves disagree, which means the error isn't `#N/A` at all. `ISNUMBER` returning `TRUE` proves `kw_text` and `SEARCH` are fine, so the fault is on the `kw_type` side. Put this in a spare cell to see the error `IFERROR` would have hidden:

```
excel =XLOOKUP(TRUE, ISNUMBER(SEARCH(kw_text, D2)), kw_type)
```

`#VALUE!` means `kw_text` and `kw_type` are different sizes — `XLOOKUP` requires them to match. Check with `=ROWS(kw_text) & " vs " & ROWS(kw_type)`, which must read `11 vs 11`. `#NAME?` means `kw_type` was never defined or is misspelled. `#REF!` means it points at deleted cells. All three are fixed in **Formulas** → **Name Manager**, not in the formula.

Note: `SEARCH` is case-insensitive, which matches the Python `re.IGNORECASE` behavior. Because `XLOOKUP` returns the first match, the row order of Table 1 is the precedence order — the same as the order of the Python schema.

After filling down, filter column J for `"Unmatched"` to verify no rows are missed.

Note that here we use the sections `kw_text` and `kw_type`, which we named before from the Reference sheet. You can find the replacement above if you do not wish to use this formatting, though most of the later functions use these, so you'll be doing a lot of extra work!

Step 4: Extract the Dose Embedded in the Order Name (Columns K and L)

Many rows have the dose written into the Order Name, such as `albumin human 5 % bottle 12.5 g`. When the Ordered Dose Amount is unusable, we pull the dose from the name before falling back to a default.

The rule: **if the name ends with a known unit, take the number immediately before it.**

Column K — Unit from Name

Add header `Name Unit` in K1. In K2:

```
=IFNA(XLOOKUP(TRUE, RIGHT(D2, LEN(name_units) + 1) = " " & name_units, name_units), "")
```

In **Google Sheets**, same change — wrap the comparison so all 7 units are tested rather than only the first:

```
=IFNA(XLOOKUP(TRUE,  
  ARRAYFORMULA(RIGHT(D2, LEN(name_units) + 1) = " " & name_units), name_units), ""  
)
```

Piece by piece:

Piece	On its own	What it contributes here
<code>LEN(name_units)</code>	Returns the character length of a string.	Across the 7-cell list this yields 7 lengths — <code>13, 11, 5, 3, 2, 2, 1</code> — one sized to each candidate unit.
<code>... + 1</code>	Adds one.	Accounts for the space that we are about to require in front of each unit.
<code>RIGHT(D2, ...)</code>	Takes N characters from the end of a string.	Handed 7 different lengths, it returns 7 different tails of the Order Name — each cut to the size of the unit it will be compared against.
<code>" " & name_units</code>	Joins text together.	Builds the 7 targets to compare against: <code>" VWF:RCo Units"</code> , <code>" Int'l Units"</code> , <code>" Units"</code> , and so on.
<code><tails> = <targets></code>	Compares two values. String comparison in Excel is case-insensitive.	Compares the two 7-item lists position by position, answering <i>does the name end with a space and then this unit?</i> seven times over.
<code>XLOOKUP(TRUE, ..., name_units)</code>	Returns the entry aligned with the first match found.	Returns the first unit that matched — which, given Table 4's ordering, is always the longest one that fits.
<code>IFNA(..., "")</code>	Supplies a fallback on <code>#N/A</code> only; other errors pass through.	No unit at the end produces <code>#N/A</code> ; blank is the signal to column L and column M that this name carries no dose. As in Step 3, a broken <code>name_units</code> range stays visible instead of masquerading as "no unit found".

Together: it asks all seven "does the name end this way?" questions in a single pass and keeps the first yes. Two details in that arrangement do real work:

- **Testing `" " & unit` rather than the unit alone** prevents a name ending in an ordinary word from being read as a unit. `injection` does not end with `" g"`, so it correctly returns blank.
- **The longest-first ordering of Table 4** is what makes `VWF:RCo Units` resolve as one unit instead of collapsing to `Units`.

Column L — Amount from Name

Add header `Name` `Amount` in **L1**. In **L2**:

```
=IF(K2 = "", "",  
    VALUE(SUBSTITUTE(TEXTAFTER(TRIM(TEXTBEFORE(D2, " " & K2, -1, 1)), " ", -1), ",", ""))  
)
```

Piece by piece:

Piece	On its own	What it contributes here
<code>IF(K2 = "", "", ...)</code>	Returns one value or another depending on a test.	A guard. No unit means no dose in the name, so it stops here and leaves the cell blank rather than letting the parsing run on a name with nothing to parse.
<code>" " & K2</code>	Joins text together.	Rebuilds the exact string column K matched on, space included, so the split happens at the unit and not at some earlier coincidence.
<code>TEXTBEFORE(D2, ..., -1, 1)</code>	Returns everything before a delimiter. <code>-1</code> means the last occurrence rather than the first; the trailing <code>1</code> makes the match case-insensitive.	Cuts the unit off the end, leaving the name up to and including the number. <code>-1</code> matters when the unit's letters also appear earlier in the name.
<code>TRIM(...)</code>	Removes leading, trailing, and repeated spaces.	Guarantees the last word really is the last word, so the next step can't come back empty on a stray trailing space.
<code>TEXTAFTER(..., " ", -1)</code>	Returns everything after a delimiter — here, after the last space.	Takes the final token of what's left, which is the number.

Piece	On its own	What it contributes here
<code>SUBSTITUTE(..., ",", "")</code>	Replaces every occurrence of one string with another.	Strips thousands separators, so <code>3,495</code> survives the next step instead of failing on the comma.
<code>VALUE(...)</code>	Converts text that looks like a number into an actual number.	Without it the result is the <i>text</i> <code>"3495"</code> , which column P cannot multiply.

Together: the pair of `TEXT...` functions is the heart of it, and neither half means much alone — `TEXTBEFORE` cuts away the unit on the right, `TEXTAFTER` then cuts away everything to the left of the number, and what survives both cuts is the dose. The rest is cleanup so the survivor is a number Excel can do arithmetic on.

Worked example. For `D2 = antihemophilic factor-vwf (AlphaNATE/VWF) injection 3,495 VWF:RCo Units`, where column K has resolved `VWF:RCo Units`:

```

TEXTBEFORE(D2, " VWF:RCo Units", -1, 1) -> antihemophilic factor-vwf (AlphaNATE/VWF) inject.
TRIM(...)                               -> (unchanged – no stray spaces)
TEXTAFTER(..., " ", -1)                  -> 3,495
SUBSTITUTE(..., ",", "")                 -> 3495
VALUE(...)                               -> 3495      <- a number, not text

```

This is the case the previous version of the guide got wrong: taking only the last word left `VWF:RCo` in front of the number, and `VALUE("VWF:RCo")` is `#VALUE!`.

If the Order Name has no dose (`albumin human 5 % bottle`), K2 is blank and L2 is blank too — exactly what we want, so the row falls through to the default.

Balfaxar is the deliberate exception. Its names put volume last (`... 1,000 Units 40 mL infusion`), so K and L come back blank. That is fine: every Balfaxar row has a usable Ordered Dose Amount, so it resolves at step 1 and never reaches name extraction.

Step 5: Resolve the Dose (Columns M, N, O)

This is the three-step fallback. For each row, the dose is resolved by priority:

1. **Ordered Dose Amount** (E) — if present *and* its unit (F) has a conversion factor for this drug.
2. **Dose from Order Name** (K and L) — if step 1 didn't resolve.
3. **Default dose** from the `Reference` sheet — if neither resolved.

Rather than re-deriving that decision in every column, we compute it **once**, in column M.

Column M — Dose Source

Add header `Dose Source` in **M1**. In **M2**:

```
=IF(AND(E2 <> "", ISNUMBER(XMATCH(J2 & "|" & F2, conv_key))), "Ordered",
IF(K2 <> "", "Name",
IF(J2 <> "Unmatched", "Default",
"")))
```

Piece by piece:

Piece	On its own	What it contributes here
<code>E2 <> ""</code>	Tests whether a cell is non-blank.	Step 1 of the fallback needs an ordered amount to exist at all. This is the cheap half of that test.
<code>J2</code> and <code>F2</code> joined by <code>&</code>	Joins text together.	Builds the same style of composite key as Table 2 — this row's drug type and its ordered unit, pipe-separated — so a single lookup can ask about a drug <i>and</i> a unit at once.
<code>XMATCH(..., conv_key)</code>	Returns the position of a value in a range, or <code>#N/A</code> if it isn't there.	Asks whether Table 2 has a conversion factor for this exact drug-and-unit pair. We never use the position itself — only whether one was found.
<code>ISNUMBER(...)</code>	<code>TRUE</code> for a number, <code>FALSE</code> for anything else including errors.	Turns "found at position 8" and " <code>#N/A</code> " into a clean <code>TRUE / FALSE</code> , and stops the <code>#N/A</code> escaping into the rest of the formula.
<code>AND(a, b)</code>	<code>TRUE</code> only when both tests pass.	The ordered dose is usable only if it exists and is convertible. Either one alone is not enough.

Piece	On its own	What it contributes here
<pre>IF(...) IF(...) IF(...)</pre>	Each returns one value or another on a test.	Nesting them makes the priority explicit: the first test that passes wins, so a row that could resolve two ways always takes the higher-quality one.
the final <code>""</code>	—	The else-of-last-resort. An <code>Unmatched</code> row has no drug type, so no default applies and the row correctly resolves to nothing.

Together: it is a priority chain, and the order of the three tests *is* the three-step fallback from the top of this step. The result is a plain label — `Ordered`, `Name`, `Default`, or blank — recording which branch the row took.

`XMATCH` against the precomputed `conv_key` column is what replaces the array formula the earlier version of this guide needed. Because column G already holds `drugtype|unit` as literal text, this is an ordinary lookup against ordinary cells; there is nothing to array-enter.

This column is worth its width. It makes the fallback visible while you scroll, it is filterable when a number looks wrong, and Step 7 counts it to show how much of the total rests on defaults rather than real data.

Column N — Resolved Unit

Add header `Resolved Unit` in **N1**. In **N2**:

```
=IFS(M2 = "Ordered", F2,
     M2 = "Name",    K2,
     M2 = "Default", XLOOKUP(J2, def_type, def_unit),
     TRUE,          "")
```

Column O — Resolved Amount

Add header `Resolved Amount` in **O1**. In **O2**:

```
=IFS(M2 = "Ordered", E2,
     M2 = "Name",    L2,
```

```
M2 = "Default", XLOOKUP(J2, def_type, def_amt),  
TRUE, "" )
```

Piece by piece — both columns share one shape, so this covers each:

Piece	On its own	What it contributes here
<code>IFS(test1, val1, test2, val2, ...)</code>	Walks the pairs left to right and returns the value beside the first test that is <code>TRUE</code> .	Replaces nested <code>IF</code> s with a flat list. Because column M already decided, these are simple equality tests rather than nested logic.
<code>M2 = "Ordered" → F2 / E2</code>	Compares a cell to text.	Reads back M's label and takes the ordered unit and amount straight from the source columns.
<code>M2 = "Name" → K2 / L2</code>	Same.	Takes the values that Step 4 parsed out of the Order Name.
<code>M2 = "Default" → XLOOKUP(J2, def_type, def_unit) / def_amt</code>	Finds the drug type in one column of Table 3 and returns the entry beside it.	Pulls the fallback dose. N and O differ <i>only</i> here, in which column of Table 3 they read.
<code>TRUE, ""</code>	A test that always passes, so its value is always reached if nothing earlier matched.	The catch-all. Without it, <code>IFS</code> returns <code>#N/A</code> when no test matches — so unmatched rows would show an error instead of a blank.

Together: N and O are deliberately the same shape and read straight down as the three fallback steps. The decision already happened in column M; these two do no thinking of their own, they only fetch the value that decision points at. That is why the earlier version's duplicated array formula could be deleted — the expensive test now happens once, in M, rather than twice here.

Step 6: Convert to Target Unit (Column P)

Add header `Converted Amount` in **P1**. In **P2**:

```
=IF(02 = "", "",
  IFNA(02 * XLOOKUP(J2 & "|" & N2, conv_key, conv_factor), "Unit not found")
)
```

Piece by piece:

Piece	On its own	What it contributes here
<code>IF(02 = "", "", ...)</code>	Returns one value or another on a test.	A guard. Nothing resolved means nothing to convert, so the cell stays blank instead of multiplying by an empty value and reporting a misleading <code>0</code> .
<code>J2</code> and <code>N2</code> joined by <code>&</code>	Joins text together.	The same composite key as column M — but built from the resolved unit in N, not the ordered unit in F. That distinction is the whole point of this step.
<code>XLOOKUP(..., conv_key, conv_factor)</code>	Finds a value in one range and returns the entry beside it in another.	Retrieves the conversion factor for this drug-and-unit pair.
<code>02 * ...</code>	Multiplies.	Applies the factor, putting every row of a given drug into one common unit so they can be summed.
<code>IFNA(..., "Unit not found")</code>	Supplies a fallback on <code>#N/A</code> only; other errors pass through.	<code>XLOOKUP</code> returns <code>#N/A</code> when the pair isn't in Table 2, and this names that problem in the cell. Other errors stay visible — a <code>#VALUE!</code> here means column O holds text rather than a number, which is a different fault needing a different fix.

Together: column N settled *which* unit this row is in; this column asks Table 2 what that unit is worth for this drug and scales the amount accordingly. Because `conv_key` is a real column of cells rather than two ranges concatenated on the fly, it is an ordinary lookup — no Ctrl+Shift+Enter.

Why the key is built from N and not F: a row whose ordered unit was `g/kg` fell through to the name or default branch, and its resolved unit is whatever *that* branch

produced. Keying off F here would look up a unit the row is no longer expressed in.

If a row shows "Unit not found", that unit has no factor for that drug — either add it to Table 2 or investigate the row.

Step 7: Summary Results

Add a summary on the existing data sheet. Use `COUNTIF` and `SUMIF` against the helper columns.

Dose Counts

```
=COUNTIF($J$2:$J$2000, "Albumin 5%")
```

Or as a table, with drug types in column A:

Drug Type	Dose Count	Total Amount
Albumin 5%	=COUNTIF(\$J\$2:\$J\$2000,A2)	=SUMIF(\$J\$2:\$J\$2000,A2,\$P\$2:\$P\$2000)
Albumin 25%	=COUNTIF(\$J\$2:\$J\$2000,A3)	=SUMIF(\$J\$2:\$J\$2000,A3,\$P\$2:\$P\$2000)
(etc.)		

Column P must be purely numeric for `SUMIF` to work — make sure no rows show "Unit not found" in the drug type you are summing.

If you want to do this in a new sheet, name the sheet with the raw and added data columns `Data`, then in that new sheet, the formulas will be:

```
=COUNTIF(Data!$J$2:$J$2000, "Albumin 5%")
```

Or as a table:

Drug Type	Dose Count	Total Amount
Albumin 5%	=COUNTIF(Data!\$J\$2:\$J\$2000,A2)	=SUMIF(Data!\$J\$2:\$J\$2000,A2,Data!\$P\$2:\$P\$2000)
Albumin 25%	=COUNTIF(Data!\$J\$2:\$J\$2000,A3)	=SUMIF(Data!\$J\$2:\$J\$2000,A3,Data!\$P\$2:\$P\$2000)
(etc.)		

How Much of This Is Real Data?

Because column M records which branch each row took, you can audit the totals:

```
=COUNTIFS($J$2:$J$2000, A2, $M$2:$M$2000, "Ordered")
=COUNTIFS($J$2:$J$2000, A2, $M$2:$M$2000, "Name")
=COUNTIFS($J$2:$J$2000, A2, $M$2:$M$2000, "Default")
```

A drug whose rows are mostly `Default` has a total that reflects your assumed dose, not the record. Check that before reporting the number.

Again, if you wanted this in a new sheet, with the data sheet named `Data`, we have the following formulas:

```
=COUNTIFS(Data!$J$2:$J$2000, A2, Data!$M$2:$M$2000, "Ordered")
=COUNTIFS(Data!$J$2:$J$2000, A2, Data!$M$2:$M$2000, "Name")
=COUNTIFS(Data!$J$2:$J$2000, A2, Data!$M$2:$M$2000, "Default")
```

Troubleshooting

Symptom	Likely cause	Fix
J shows <code>Unmatched</code>	Order Name matches no keyword	Add a row to Table 1 (keywords) — no formula edit needed
P shows <code>Unit not found</code>	The (drug type, resolved unit) pair has no factor	Add a row to Table 2, then fill the <code>key</code> formula down to cover it

Symptom	Likely cause	Fix
M shows <code>Name</code> when you expected <code>Ordered</code>	The unit in F is not in Table 2 for that drug (e.g. <code>g/kg</code>)	Expected for weight-based orders — they can't convert without patient weight
K is blank but the name clearly has a dose	The name doesn't end with the unit (Balfaxar), or the unit is missing from Table 4	Add the unit to Table 4, keeping the list longest-first
L shows <code>#VALUE!</code> or <code>#N/A</code>	The token before the unit isn't a number, so the name isn't <code><number> <unit></code>	Inspect that Order Name; if it's a new format, it needs its own handling
A total looks far too low	Rows resolved to <code>Default</code> and the default amount is still blank	Fill in Table 3 — blank defaults contribute 0
A formula returns <code>#NAME?</code>	A named range wasn't defined, or the name is misspelled	Formulas → Name Manager and check all eight names from Step 2
A lookup returns <code>#VALUE!</code>	The two ranges given to <code>XLOOKUP</code> are different sizes; it requires them to match	Check with <code>=ROWS(kw_text) & " vs " & ROWS(kw_type)</code> — it must read <code>11 vs 11</code>
J says <code>Unmatched</code> but <code>ISNUMBER</code> is <code>TRUE</code>	Not a matching failure at all — a real error on the <code>kw_type</code> side, formerly hidden	Drop the <code>IFNA</code> wrapper to reveal the error; see the warning in Step 3